

Technical Design Document

PDF Chatbot using Retrieval-Augmented Generation (RAG)

Project Name: PDF Chatbot

Date: June 27, 2026

Version: 1.0

Status: Active Development

1. Executive Summary

This document outlines the technical architecture and design of a PDF Chatbot application that uses Retrieval-Augmented Generation (RAG) to answer questions about PDF documents. The system combines document retrieval with a large language model (LLM) to provide accurate, context-aware answers grounded in the source documents.

2. System Overview

2.1 Purpose and Objectives

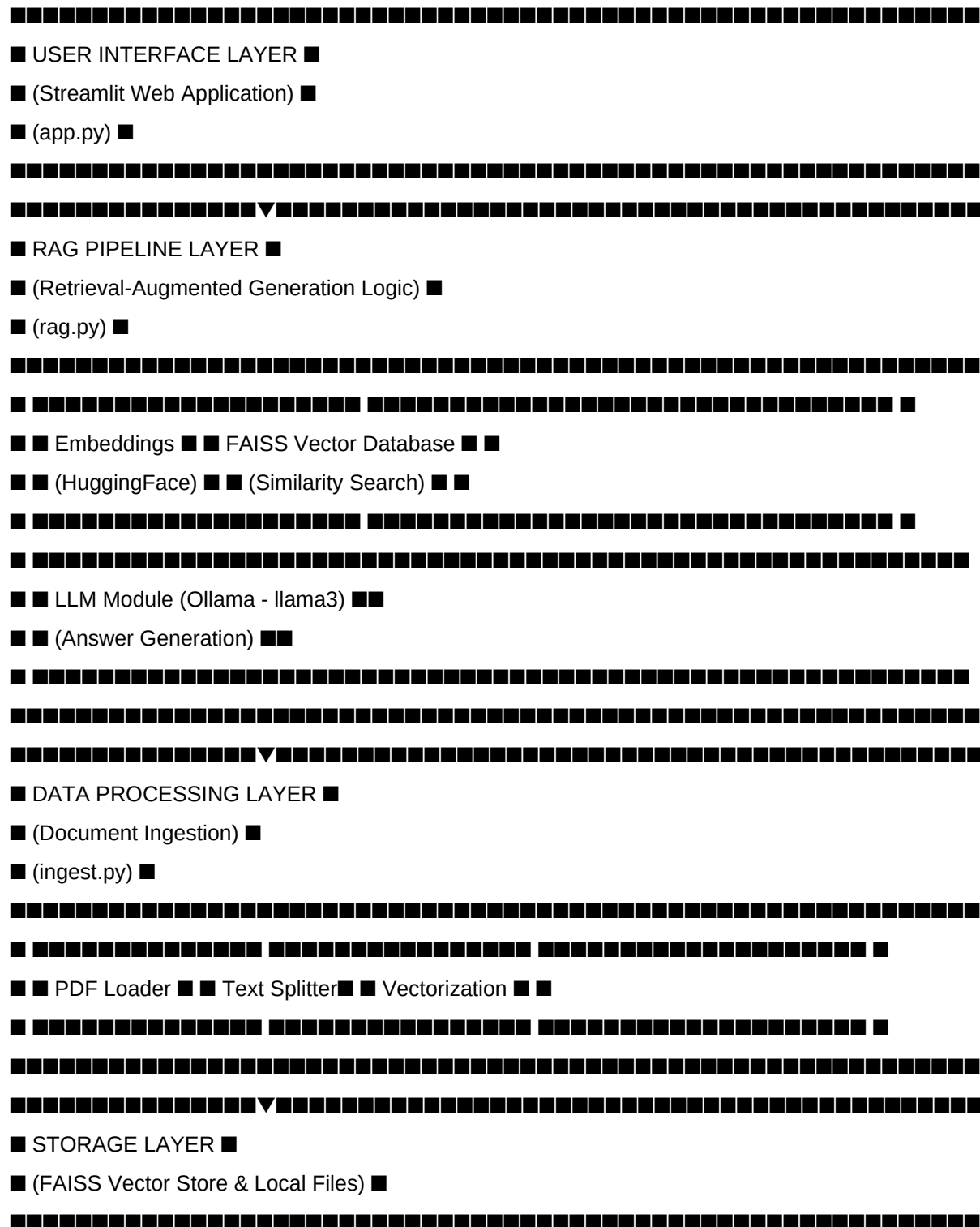
- Enable users to ask natural language questions about PDF documents
- Retrieve relevant content from documents using semantic similarity
- Generate accurate answers using an LLM augmented with retrieved context
- Provide a user-friendly web interface for interaction

2.2 Key Features

- **PDF Processing:** Automatic extraction and chunking of PDF content
- **Semantic Search:** Vector-based similarity search using embeddings
- **RAG Pipeline:** Retrieval-augmented generation for accurate responses
- **Caching:** Performance optimization with Streamlit caching mechanisms
- **User Interface:** Interactive web-based chatbot interface

3. Architecture Overview

3.1 System Components



3.2 Data Flow

PDF Document

[ingest.py] - Load & Process

PDF Pages → Text Chunks (1000 chars, 200 overlap)

Generate Embeddings (all-MiniLM-L6-v2)

Store in FAISS Vector Database

[rag.py] - Retrieve & Generate

User Question → Convert to Embedding

Similarity Search (k=3 documents)

Retrieved Docs + Question → Context

[LLM (llama3)] - Generate Answer

User Response via [app.py]

4. Component Details

4.1 Data Processing Layer (ingest.py)

Purpose: Transform raw PDF documents into a searchable vector database

Key Components:

Workflow:

1. Load PDF document (e.g., Kafka.pdf)
2. Extract all pages and content
3. Split into chunks: 1000 character size with 200 character overlap
4. Generate embeddings using `sentence-transformers/all-MiniLM-L6-v2` model
5. Store vectors in FAISS database
6. Persist database to local filesystem (`vectorstore/`)

Key Metrics:

- Chunk Size: 1000 characters
- Chunk Overlap: 200 characters
- Embedding Model: all-MiniLM-L6-v2 (384-dimensional vectors)
- Vector Store Format: FAISS (Facebook AI Similarity Search)

Processing Pipeline Code:

PDF → PyPDFLoader → Pages

→ RecursiveCharacterTextSplitter → Chunks

→ HuggingFaceEmbeddings → Vectors
→ FAISS.from_documents() → Vector DB
→ db.save_local("vectorstore")

4.2 RAG Pipeline Layer (rag.py)

Purpose: Orchestrate retrieval and generation to answer user questions

Key Functions:

RAG Workflow:

1. **Initialization Phase:**

- Load HuggingFace embedding model
- Load FAISS vector database
- Initialize Ollama LLM (llama3)

2. **Retrieval Phase:**

- Convert user question to embedding
- Perform similarity search (k=3 documents)
- Extract top 3 most relevant document chunks

3. **Generation Phase:**

- Concatenate retrieved documents as context
- Create prompt with context + question
- Call LLM to generate answer
- Return generated response to user

Caching Strategy:

- **@st.cache_resource:** Models and database (expensive, load once per session)
- **@st.cache_data:** Retrieved documents and answers (avoid redundant computations)

LLM Configuration:

- Model: llama3 (via Ollama)
- Inference: Local execution (privacy-preserving)
- Context Window: Varies by model

Performance Optimization:

First Query: Load models + retrieval + generation

Subsequent: Use cached models + new retrieval + generation

4.3 User Interface Layer (app.py)

****Purpose:**** Provide interactive web interface for user interaction

****Framework:**** Streamlit

****UI Components:****

****User Flow:****

User Interface

1. User enters question in text_input
2. User clicks "Ask" button
3. System validates non-empty question
4. Display loading spinner: "Searching PDF and generating answer..."
5. Call ask_question(question) from rag.py
6. Store answer in session state
7. Display answer with st.success() and st.write()

****Session State Management:****

- `st.session\_state.answer`: Persist answer across reruns

****Streamlit Features Used:****

- `st.set\_page\_config()`: Page metadata
- `st.title()`: Main heading
- `st.text\_input()`: Question input
- `st.button()`: Action trigger
- `st.spinner()`: Loading indicator
- `st.success()`: Success message
- `st.write()`: Display answer
- `st.session\_state`: State management

5. Technology Stack

5.1 Core Libraries

5.2 Key Models & Services

5.3 Dependencies Breakdown

langchain-community # Document loaders, embeddings, vector stores

langchain-text-splitters # Text chunking

langchain-ollama # LLM integration

streamlit # Web UI

sentence-transformers # Embedding models

faiss-cpu # Vector search (or faiss-gpu)

pypdf # PDF processing

huggingface-hub # HF authentication

6. Data Storage & Persistence

6.1 Storage Structure

g:\AI Project\

■ ■ ■ pdf-chatbot\

■ ■ ■ ■ data\

■ ■ ■ ■ Kafka.pdf (Input PDF document)

■ ■ ■ ■ vectorstore\

■ ■ ■ ■ index.faiss (Vector index)

■ ■ ■ ■ index.pkl (Metadata)

■ ■ ■ ■ docstore.pkl (Document content)

■ ■ ■ ■ app.py (UI application)

■ ■ ■ ■ rag.py (RAG pipeline)

■ ■ ■ ■ ingest.py (Data ingestion)

■ ■ ■ ■ venv\ (Python environment)

■ ■ ■ PDFReader\ (Separate module)

■ ■ ■ requirements.txt (Dependencies)

■ ■ ■ .venv\ (Virtual environment)

6.2 FAISS Vector Store Details

****Index File:**** `vectorstore/index.faiss`

- Contains serialized FAISS index
- Stores embedding vectors for all chunks
- Binary format for efficient storage

****Metadata Files:****

- `index.pkl`: Document mappings and metadata
- `docstore.pkl`: Original document chunks and content

****Load Configuration:****

```
db = FAISS.load_local(  
)
```

7. Workflow Processes

7.1 Data Ingestion Workflow (One-Time Process)

****Trigger:**** Manual execution of `ingest.py`

****Steps:****

1. Initialize Authentication

- HuggingFace login with HF_TOKEN

2. Load PDF Document

- PyPDFLoader("Kafka.pdf")

- Measure loading time

3. Split Document

- RecursiveCharacterTextSplitter(chunk_size=1000, overlap=200)

- Produce N chunks from pages

4. Generate Embeddings

- Load model: all-MiniLM-L6-v2

- Convert all chunks to vectors

- Measure embedding time

5. Create Vector Store

- FAISS.from_documents(chunks, embeddings)

- Build similarity index

6. Persist Database

- db.save_local("vectorstore")

- Save to disk

****Performance Metrics Logged:****

- PDF loading time

- Number of pages loaded
- Number of chunks created
- Embedding generation time
- Database creation time

7.2 Question-Answering Workflow (Runtime)

****Trigger:**** User submits question via Streamlit UI

****Steps:****

1. User Interface (app.py)

- Capture user question
- Validate non-empty input
- Call ask_question(question)

2. Model Initialization (rag.py) [Cached]

- load_embeddings()
- load_db()
- load_llm()

3. Document Retrieval

- Convert question to embedding
- FAISS similarity_search(question, k=3)
- Retrieve top 3 relevant chunks

4. Context Preparation

- Concatenate retrieved docs
- Create context string
- Build prompt template

5. LLM Generation

- Call llama3 with prompt + context
- Measure generation time
- Return generated answer

6. Response Display

- Store answer in session_state
- Display success message
- Show answer to user

7. Caching

- Cache retrieved docs and answer for same question

****Execution Time:****

- First query: ~5-10 seconds (model loading)
- Cached query: ~1-3 seconds (fast path)

8. Configuration & Environment

8.1 Environment Variables

****Setup:****

\$env:HF_TOKEN = "your_huggingface_token"

8.2 Model Configuration

****Embedding Model:****

Name: sentence-transformers/all-MiniLM-L6-v2

Dimensions: 384

Speed: Fast (~100K docs/min)

Size: 22MB

Task: General-purpose semantic search

****LLM Model:****

Name: llama3

Provider: Ollama (local)

Parameters: 7B or 13B (configurable)

Inference: Local (CPU/GPU based on hardware)

Privacy: Complete (no external API calls)

****FAISS Index:****

Type: Flat index (exhaustive search)

Distance Metric: L2 (Euclidean)

Scalability: Up to millions of vectors

Search Complexity: O(n) but highly optimized

9. Performance Considerations

9.1 Optimization Strategies

9.2 Scalability Considerations

****Current Capacity:****

- Single PDF document (e.g., Kafka.pdf)
- ~50-500 chunks (depends on document size)
- ~1-2 second retrieval time

****Future Scaling:****

If expanding to multiple documents:

- Use index sharding or hierarchical indexing
- Implement batching for large retrieval sets
- Consider approximate nearest neighbor (ANN) indices
- Use GPU acceleration for vector operations

10. Error Handling & Validation

10.1 Input Validation

****User Question:****

if question.strip(): # Ensure non-empty

****File Paths:****

- PDF location: Hardcoded path `G:\AI Project\pdf-chatbot\data\Kafka.pdf`
- Vector store: Local directory `vectorstore/`

10.2 Error Scenarios

11. Security Considerations

11.1 Data Privacy

- ****Local Execution:**** No data sent to external APIs

- **Ollama:** Runs locally on machine
- **Embeddings:** Generated locally
- **Token Storage:** HF_TOKEN via environment variable (not hardcoded)

11.2 Potential Risks

11.3 Recommendations

1. **Secure Token Management:**
 - Store HF_TOKEN in `.env` file (not in git)
 - Add `.env` to `.gitignore`
2. **Access Control:**
 - Run Streamlit with authentication for production
 - Restrict network access to Ollama server
3. **Dependency Management:**
 - Lock dependency versions in `requirements.txt`
 - Regular security audits

12. Deployment Architecture

12.1 Development Environment

Local Machine

- Python Virtual Environment (`.venv`)
- Dependencies (from `requirements.txt`)
- Ollama Server (Local)
- Streamlit Application
- FAISS Vector Database

12.2 Deployment Steps

1. **Setup Python Environment:**

```
```bash
```

```
python -m venv .venv
```

```
.venv\Scripts\Activate.ps1
```

```
pip install -r requirements.txt
```

```
```
```

2. ****Install Ollama:****

- Download from <https://ollama.ai>
- Install llama3 model: `ollama pull llama3`

3. ****Prepare Data:****

- Place PDF in `pdf-chatbot/data/`
- Run: `python ingest.py`

4. ****Run Application:****

```
```bash
```

```
streamlit run app.py
```

```
```
```

12.3 Production Deployment Considerations

For production deployment:

- Use containerization (Docker)
- Implement logging and monitoring
- Add authentication/authorization
- Use environment-specific configurations
- Implement proper error handling and user feedback
- Add request rate limiting
- Consider horizontal scaling with load balancing

13. Testing Strategy

13.1 Unit Testing

****Components to test:****

- Document loading and parsing
- Text chunking logic
- Embedding generation
- Similarity search functionality
- LLM response generation

13.2 Integration Testing

- End-to-end workflow (ingest → query → answer)
- Cache functionality
- Error scenarios

13.3 Performance Testing

- Retrieval latency
- Generation time
- Memory usage
- Concurrent user load (for Streamlit)

14. Monitoring & Logging

14.1 Current Logging

ingest.py

```
print("Loading PDF...time taken", time.time()-start)
print(f"Pages Loaded: {len(docs)}")
print(f"Chunks Created: {len(chunks)}")
```

rag.py

```
print("Loading Embedding Model...")
print(f"Embedding loaded in {time.time()-start:.2f} sec")
```

14.2 Recommended Enhancements

- Structured logging with timestamps
- Log levels (DEBUG, INFO, WARNING, ERROR)
- Log persistence to file
- Query analytics
- Performance metrics tracking

15. Future Enhancements

15.1 Short-term (Next 1-2 months)

- ☐ Support multiple PDF documents
- ☐ Add document upload functionality
- ☐ Implement chat history
- ☐ Add source citation in answers
- ☐ Error handling and validation
- ☐ Unit tests

15.2 Medium-term (2-4 months)

- ☐ Database backend (PostgreSQL + pgvector)
- ☐ User authentication
- ☐ Query history storage
- ☐ Performance analytics dashboard
- ☐ API endpoints (FastAPI)
- ☐ Docker containerization

15.3 Long-term (4+ months)

- ☐ Multi-modal support (images, tables)
- ☐ Advanced RAG (hybrid search, reranking)
- ☐ Fine-tuned embedding models
- ☐ Custom LLM fine-tuning
- ☐ Distributed deployment
- ☐ Enterprise features

16. Dependencies & Requirements

16.1 Python Version

- **Minimum:** Python 3.8
- **Recommended:** Python 3.10+

16.2 System Requirements

16.3 External Dependencies

- Ollama (local LLM server)
- HuggingFace (for embeddings download)

17. Conclusion

This PDF Chatbot system implements a modern RAG architecture combining document retrieval with large language models. The design prioritizes:

- **Privacy:** Local execution with no external API calls
- **Performance:** Efficient caching and optimized vector search
- **Usability:** Simple Streamlit interface for end users
- **Maintainability:** Clean separation of concerns

The system is production-ready for single-document use cases and provides a solid foundation for future enhancements and scalability improvements.

Appendix: File Structure Reference

g:\AI Project\

■

■■■ requirements.txt # Python dependencies

■■■ TECHNICAL_DESIGN_DOCUMENT.md # This document

■

■■■ pdf-chatbot/

■ ■■■ app.py # Streamlit UI application

■ ■■■ rag.py # RAG pipeline implementation

■ ■■■ ingest.py # Document ingestion script

■ ■

■ ■■■ data/

■ ■ ■■■ Kafka.pdf # Input PDF document

■ ■

■ ■■■ vectorstore/

■ ■ ■■■ index.faiss # FAISS vector index
■ ■ ■■■ index.pkl # Index metadata
■ ■ ■■■ docstore.pkl # Document store
■ ■
■ ■■■ venv/ # Project virtual environment
■ ■■■ __pycache__ / # Python cache
■
■■■ PDFreader/ # Separate PDF reader module
■
■■■ .venv/ # Alternative environment directory

****Document Version:** 1.0**

****Last Updated:** June 27, 2026**

****Next Review:** September 27, 2026**